



DevOps Home Assignment

Candidate Assessment Task

Document name: DevOps Home Assignment

This document, along with any attached documents, is classified and contains sensitive business information and/or proprietary property of the company and/or its clients. It is authorized for viewing solely by employees and its clients, subject to written approval and the signing of a Non-Disclosure Agreement (NDA).



Compie — DevOps Home Assignment

Time: about 4–6 hours

AI & tools: Using AI is allowed and encouraged. Search engines, docs, and assistants are fine — just don't paste secrets into public places.

Please note that this assignment can be done with small / free-tier-friendly AWS resources. Tear everything down when you finish to avoid ongoing costs.

Build everything from scratch — we do not provide a starter project.

Submission

Hand in a zip with everything you built — application, Dockerfile, Terraform, pipeline definitions, README, screenshots, and any extras. Keeping your git history inside the zip helps us see how you worked.

You do not need to leave the AWS environment running after you finish. Include a simple browser screenshot of the application while it was live (for example `/`, and optionally `/health`).

End goal

The end goal is to serve a small application on the public internet by:

- provisioning AWS infrastructure with Terraform (Terragrunt optional)
- dockerizing the app and running it on EC2 behind an Auto Scaling Group, reached via a public load balancer while compute stays private
- releasing through a CI/CD pipeline to a `dev` environment
- including basic logging, monitoring/alerting, and encryption (KMS) so the stack is operable — not only “it starts”

Use a minimal app of your choice. It must expose `/` and `GET /health`. Keep app logic minimal; this assignment is about delivery and operations.



Guidelines — Docker

- Provide a `Dockerfile` (and whatever else you need for a clean local run)
- Show that the image runs locally and that `/` and `/health` work

We will review Dockerfile practices, not only that the container starts.

Guidelines — Infrastructure (Terraform)

Networking and compute

- VPC with public and private subnets
- Run the container on EC2 instances in an Auto Scaling Group in private subnets (no public IPs on the instances)
- A public Application Load Balancer fronts the ASG
- ASG defined in Terraform: launch template, target group, health checks, sensible min / desired / max
- In the README, briefly explain how you manage capacity / how you would scale
- Do not use ECS or EKS / Kubernetes for this task

Make sure you have security in mind when designing the architecture (private compute, least-privilege security groups).

Image, config, and IAM

- The image lives in Amazon ECR
- Configuration that shouldn't live in the image (message, token, DB connection details, or similar) lives in a managed secrets/config store on AWS; the running service consumes it
- IAM is split by responsibility. At minimum:
 - an instance profile that can pull from ECR, read the app secret/config, write logs, and use SSM



- a CI/CD identity that can push to ECR and trigger a rollout — but not read the app secrets
- Avoid one oversized “admin” identity

Data store

- Provision a managed database on AWS (engine of your choice — e.g. RDS or DynamoDB)
- Keep it private — not publicly reachable; only the application instances should reach it
- Do not hardcode credentials or connection strings; use the secrets/config approach above
- The running app should prove it can talk to the DB (for example `/health` checks the DB, or `/` reads/writes a trivial record)
- Prefer a small / free-tier-friendly size; destroy when done

Logging, encryption, and monitoring

- Application / instance logs go to CloudWatch Logs (we care that logs are reachable and usable for basic traceability)
- Encrypt sensitive data at rest with KMS — at least the database and the secrets/config store. AWS-managed keys are fine
- Define monitoring/alerting on infrastructure or service state (for example unhealthy ALB targets, instance health, or database signals — you choose what matters)
- Send those alerts to an endpoint. The endpoint may be a placeholder — you do not need a real paging product. Configure the wiring properly (topic/subscription, webhook, or equivalent) and document what would receive alerts in a real setup

`terraform apply` (plus any bootstrap you document) should result in a working public URL.



Guidelines — CI/CD

Implement a full CI/CD pipeline with a tool of your choice (GitHub Actions, GitLab CI, Bitbucket Pipelines, or Jenkins).

- The pipeline should build the image, push it to ECR, and deploy/release it to your `dev` environment (ASG behind the ALB) so a change can go from commit to running app without manual console steps
- A single branch is enough. How you trigger the release (push, tag, or manual) is up to you — document it
- Do not put secrets in the zip
- Any tests, scans, or extra checks are welcome, but not required

Bonus (nice to have — not required)

- A CloudWatch (or similar) dashboard — include a screenshot in the zip
- Architecture diagrams or network sketches
- Skills, notes, `.md` files, prompts, or other artifacts showing how you used AI — useful for our assessment
- A customer-managed KMS key; encrypted EBS on the launch template
- HTTPS on the public endpoint
- Dynamic ASG scaling policy (e.g. target-tracking on CPU)
- PR / pre-merge checks; a second environment

Evaluation criteria

1. Correctness — App reachable; health; DB wired; apply path works
2. Elegance / maintainability — Clear, maintainable Terraform and project layout
3. Security — Private networking, least-privilege IAM, secrets handling, KMS encryption at rest
4. Logging & monitoring — CloudWatch logs; alerting wired to a placeholder endpoint
5. Pipeline — Full CI/CD: build, push, deploy to `dev`



6. Documentation — README someone else can follow (apply/destroy, pipeline trigger, capacity)

A simple design that works and is secure beats a large incomplete one.

Good luck.